

INDUSTRIAL TRAINING ASSIGNMENT

Topic 1: Functions

(15 Questions with Answers)

Topic 2: Function Calls

(15 Questions with Answers)

Topic 3: Recursion

(15 Questions with Answers)

Submitted By:

Name: Kirti Soni

Branch: B.Tech AIDS(B)

INDUSTRIAL TRAINING ASSIGNMENT

TOPIC 1: FUNCTIONS

Q1. What is a function in C?

Answer:

A function is a self-contained block of code that performs a specific task. It helps divide a large program into smaller and manageable parts.

Q2. What is the syntax of a function definition?

Answer:

The general syntax is:

```
return_type function_name(parameters)
{
    statements;
}
```

Example:

```
int add(int a, int b)
{
    return a+b;
}
```

Q3. What is a function prototype?

Answer:

A function prototype tells the compiler about the function name, return type, and parameters before the function is called.

Example:

```
int add(int, int);
```

Q4. Differentiate between function declaration and function definition.

Answer:

Function declaration only informs the compiler about the function, whereas function definition contains the actual implementation of the function.

Q5. What are the advantages of functions?

Answer:

1. Code reusability
2. Easy debugging
3. Better readability
4. Reduced code duplication
5. Easy maintenance

Q6. What are actual and formal parameters?

Answer:

Actual parameters are the values passed during a function call.

Formal parameters are the variables that receive those values in the function definition.

Q7. What is a return statement?

Answer:

A return statement terminates the execution of a function and returns a value to the calling function.

Q8. Can a function return multiple values?

Answer:

No. A function can directly return only one value. Multiple values can be returned using pointers, arrays, or structures.

Q9. What is a void function?

Answer:

A void function is a function that does not return any value.

Example:

```
void display()
{
printf("Hello");
}
```

Q10. What is recursion?

Answer:

Recursion is a process in which a function calls itself repeatedly until a base condition is satisfied.

Q11. What is function overloading?

Answer:

Function overloading allows multiple functions with the same name but different parameter lists. It is supported in C++ but not in C.

Q12. What is the scope of a local variable?

Answer:

A local variable can be accessed only within the function where it is declared.

Q13. What is a global variable?

Answer:

A variable declared outside all functions is called a global variable and can be accessed throughout the program.

Q14. What happens if a non-void function does not return a value?

Answer:

It may lead to undefined behavior and compiler warnings.

Q15. Write a function to calculate the square of a number.

Answer:

```
int square(int n)
{
return n*n;
}
```

TOPIC 2: FUNCTION CALLS

Q1. What is a function call?

Answer:

A function call transfers control from the calling function to the called function for execution.

Q2. What is Call by Value?

Answer:

In Call by Value, a copy of the actual argument is passed to the function. Changes made inside the function do not affect the original variable.

Q3. What is Call by Reference?

Answer:

In Call by Reference, the address of the variable is passed. Any changes made inside the function affect the original variable.

Q4. Predict the output:

```
void fun(int x)
{
x=x+10;
}
int main()
{
int a=5;
fun(a);
printf("%d",a);
}
```

Answer:

Output: 5

Reason: Call by Value is used.

Q5. Predict the output:

```
int fun(int x)
{
return x*x;
}
int main()
{
printf("%d",fun(4));
}
```

Answer:

Output: 16

Q6. What is nested function calling?

Answer:

Calling one function inside another function call is known as nested function calling.

Q7. What is parameter passing?

Answer:

Parameter passing is the process of supplying values to a function during a function call.

Q8. What is stack memory used for during function calls?

Answer:

Stack memory stores:

1. Function parameters
2. Return address
3. Local variables

Q9. Predict the output:

```
int f(int x)
{
return x+1;
}
int main()
{
printf("%d",f(f(2)));
}
```

Answer:

Output: 4

Q10. What is a return address?

Answer:

A return address is the memory location to which control returns after a function finishes execution.

Q11. Predict the output:

```
int fun(int a,int b)
{
return a+b;
}
```

```
}  
int main()  
{  
printf("%d",fun(5,10));  
}
```

Answer:

Output: 15

Q12. What happens when a function is called?

Answer:

1. Control transfers to the function.
2. Parameters are passed.
3. Function executes.
4. Return value is generated.
5. Control returns to the calling function.

Q13. What is function call overhead?

Answer:

The extra execution time required for parameter passing, stack allocation, and returning control is called function call overhead.

Q14. What is an activation record?

Answer:

An activation record is a data structure stored in stack memory for each function call. It contains local variables, parameters, and return addresses.

Q15. Why are function calls important?

Answer:

1. Improve modularity
2. Reduce code duplication
3. Improve readability
4. Simplify maintenance

TOPIC 3: RECURSION

Q1. What is recursion?

Answer:

Recursion is a programming technique in which a function calls itself.

Q2. What are the essential components of recursion?

Answer:

1. Base Case
2. Recursive Case

Q3. Write a recursive factorial function.

Answer:

```
int fact(int n)
{
    if(n==0)
        return 1;
    return n*fact(n-1);
}
```

Q4. Find the output:

```
void fun(int n)
{
    if(n==0)
        return;
    printf("%d ",n);
    fun(n-1);
}
fun(3);
```

Answer:

Output: 3 2 1

Q5. Find the output:

```
void fun(int n)
{
    if(n==0)
        return;
    fun(n-1);
    printf("%d ",n);
}
fun(3);
```

Answer:

Output: 1 2 3

Q6. What is direct recursion?

Answer:

When a function directly calls itself, it is called direct recursion.

Q7. What is indirect recursion?

Answer:

When two or more functions call each other recursively, it is called indirect recursion.

Example:

fun1() calls fun2()

fun2() calls fun1()

Q8. What is tail recursion?

Answer:

If the recursive call is the last statement executed in the function, it is called tail recursion.

Q9. What is a recursion tree?

Answer:

A recursion tree is a graphical representation showing all recursive calls and their relationships.

Q10. What is stack overflow in recursion?

Answer:

Stack overflow occurs when too many recursive calls exhaust the available stack memory.

Q10. What is stack overflow in recursion?

Answer:

Stack overflow occurs when too many recursive calls exhaust the available stack memory.

Q11. What is the output?

```
int fun(int n)
{
    if(n==1)
        return 1;
    return n+fun(n-1);
}
printf("%d",fun(4));
```

Answer:

Output: 10

Explanation:

$$4 + 3 + 2 + 1 = 10$$

Q12. What are the advantages of recursion?

Answer:

1. Shorter code
2. Easier implementation of tree algorithms
3. Elegant solutions for complex problems

Q13. What are the disadvantages of recursion?

Answer:

1. Higher memory consumption
2. Slower execution
3. Risk of stack overflow

Q14. Write a recursive Fibonacci function.

Answer:

```
int fib(int n)
{
    if(n<=1)
        return n;
    return fib(n-1)+fib(n-2);
}
```

Q15. What is the time complexity of the Fibonacci recursive algorithm?

Answer:

The time complexity is $O(2^n)$ because each function call generates two additional recursive calls.